Accuknox Tasks

Task 1: API Data Retrieval and Storage

Fetch data from an external REST API, store it in a local SQLite database, and display the retrieved data. The API provides a list of books in JSON format with attributes like title, author, and publication year.

Solution

Python

```python
import requests as rq
import sqlite3 as sq

def retrieve():
    link = "https://openlibrary.org/subjects/fiction.json?limit=10"
    try:
        res = rq.get(link, timeout=10)
        res.raise_for_status()
        data = res.json()
        books = []

        for item in data.get("works", []):
            books.append({
                "title": item.get("title", "Unknown"),
                "author": item.get("authors", [{}])[0].get("name", "") if item.get("authors") else "Unknown",
                "year": item.get("first_publish_year")
            })
        return books
    except rq.RequestException as error:
        print(f"Error retrieving data: {error}")
        return []

def create_database():
    connection = sq.connect("lib.db")
    cursor = connection.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS lib(
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            title TEXT,
            author TEXT,
            publication_year INTEGER
        )
    """)
    connection.commit()
    return connection

def store_books(connection, books):
    """Insert book records into SQLite"""
    cursor = connection.cursor()
    for book in books:
        cursor.execute("""
            INSERT INTO lib (title, author, publication_year)
            VALUES (?, ?, ?)
        """, (book["title"], book["author"], book["year"]))
    connection.commit()

def display_books(connection):
    cursor = connection.cursor()
    cursor.execute("""
        SELECT id, title, author, publication_year
        FROM lib
    """)
    books = cursor.fetchall()
    print("\nThese are the books:")
    for record in books:
```

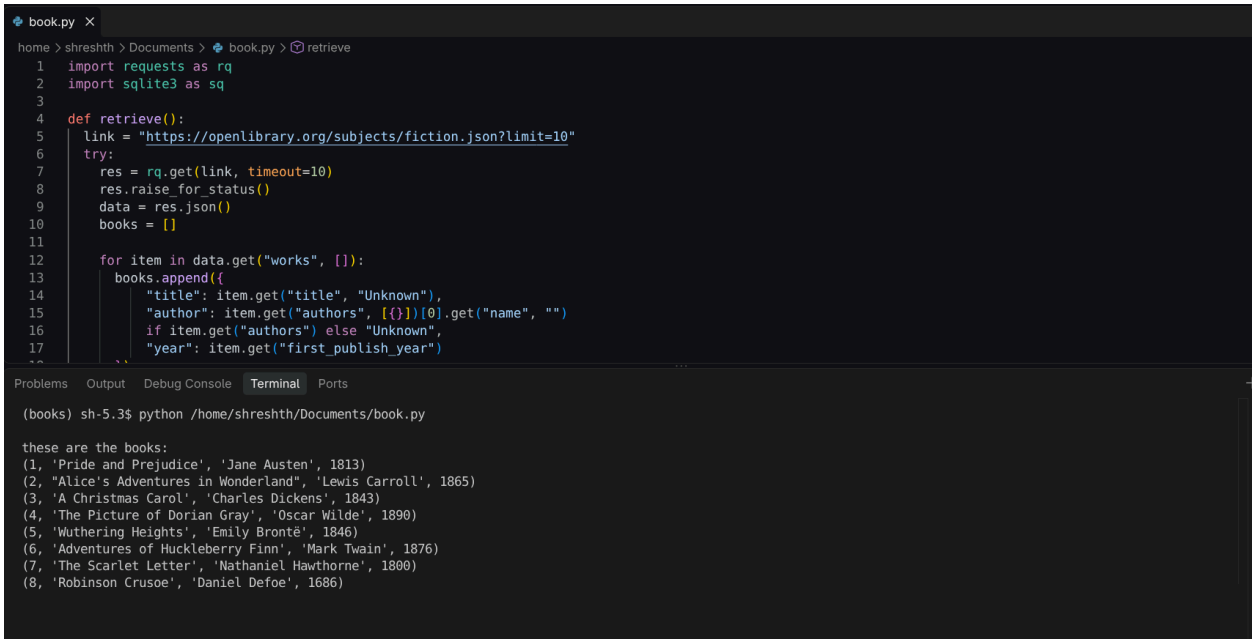
```
        print(record)

def main():
    books = retrieve()
    if not books:

                print("No books were retrieved.")
        return

    connection = create_database()
    try:
        store_books(connection, books)
        display_books(connection)
    finally:
        connection.close()

if __name__ == "__main__":
    main()
```



Task 2: Data Processing and Visualization

Fetch student test score data from an API, calculate the average score, and create a bar chart to visualize the results.

Solution

Python

```python
import requests
import pandas as pd
import matplotlib.pyplot as plt

def fetch_student_score():
    url = "https://api.example.com/student_scores"
    try:
        response = requests.get(url, timeout=12)
        response.raise_for_status()
        return response.json()
    except requests.RequestException as error:
        print("API request failed.")
        return None

def main():
    data = fetch_student_score()
    if data is None:
        print("dummy data used.")
        data = [
            {"name": "Aandu", "score": 12},
            {"name": "Myandu", "score": 34},
            {"name": "Syandu", "score": 36},
            {"name": "Chirandu", "score": 11},
            {"name": "Akhanadanu", "score": 72}
        ]

    df = pd.DataFrame(data)
    if not {"name", "score"}.issubset(df.columns):

                raise ValueError("Data must contain 'name' and 'score' columns.")

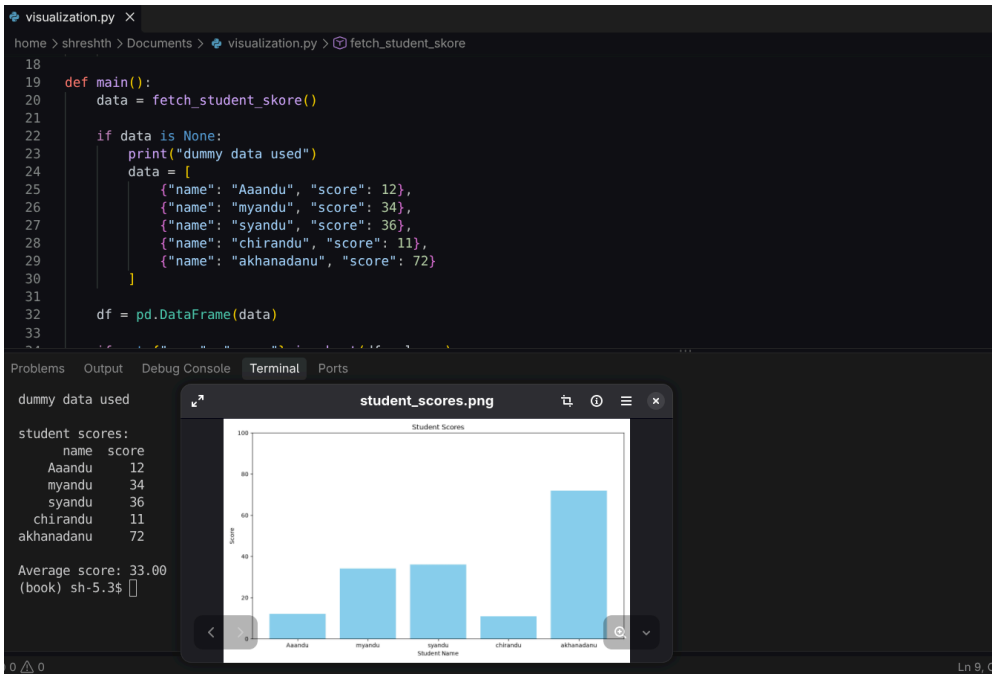
    df["score"] = pd.to_numeric(df["score"], errors="raise")
    if df.empty:
        print("No data found.")
        return

    avg_score = df["score"].mean()
    print("\nStudent Scores:")
    print(df.to_string(index=False))
    print(f"\nAverage score: {avg_score:.2f}")

    plt.figure(figsize=(10, 6))
    plt.bar(df["name"], df["score"], color="skyblue")
    plt.xlabel("Student Name")

         plt.ylabel("Score")
    plt.title("Student Scores")
    plt.ylim(0, 100)
    plt.tight_layout()
    plt.savefig("student_scores.png")
    plt.show()
```

```python
if __name__ == "__main__":
    main()
```



Task 3: CSV Data Import to a Database

Read data from a CSV file containing user information (e.g., name, email) and insert it into a SQLite database.

Solution

Python

```python
import csv
import sqlite3

with open("users.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["name", "email"])
    writer.writerow(["monuz", "monuz@example.com"])
    writer.writerow(["loluz", "loluz@example.com"])
    writer.writerow(["goluz", "goluz@example.com"])

print("success")

connection = sqlite3.connect("users.db")
cursor = connection.cursor()
cursor.execute("""
    CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT,
        email TEXT
    )
""")


with open("users.csv", "r", newline="") as file:

         reader = csv.DictReader(file)
    for row in reader:
        cursor.execute(
            "INSERT INTO users (name, email) VALUES (?, ?)",
            (row["name"], row["email"])
        )

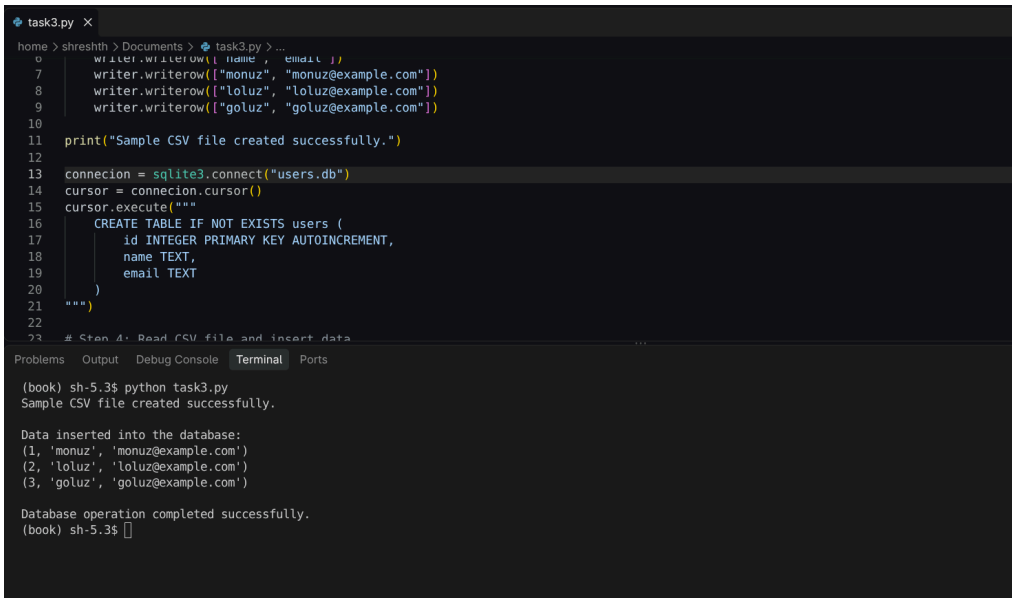
connection.commit()

# Step 4: Display inserted data
cursor.execute("SELECT * FROM users")
records = cursor.fetchall()

print("\nData inserted into database:")
for record in records:

        print(record)


connection.close()
print("\nDatabase operation completed successfully.")
```


task3.py
home > shreshth > Documents > task3.py > ...
writer.writerow(["name", "email"])
writer.writerow(["monuz", "monuz@example.com"])
writer.writerow(["loluz", "loluz@example.com"])
writer.writerow(["goluz", "goluz@example.com"])
print("Sample CSV file created successfully.")
connecion = sqlite3.connect("users.db")
cursor = connecion.cursor()
cursor.execute("""
CREATE TABLE IF NOT EXISTS users (
id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT,
email TEXT
)
""")
Step 4: Read CSV file and insert data
Problems
Output
Debug Console
Terminal
Ports
(book) sh-5.3$ python task3.py
Sample CSV file created successfully.
Data inserted into the database:
(1, 'monuz', 'monuz@example.com')
(2, 'loluz', 'loluz@example.com')
(3, 'goluz', 'goluz@example.com')
Database operation completed successfully.
(book) sh-5.3$

Task 4: Code link

> Send a link to the most complex Python code you have written

Code link github

Task 5: Code link

> Send a link to the most complex database code you have written

code link drive

It is part of genmate project

1. Where would you rate yourself on (LLM, Deep Learning, AI, ML). A, B, C [A = can code independently; B = can code under supervision; C = have little or no understanding]

   Solution:

   I would rate myself A, I can independently write code, utilize AI agents, MCP servers, git for version tracking and restoring code whenever something goes wrong, write meaningful PR messages and utilize Sonarqube for PR messages and formatting. I not only have an understanding of how LLMs work but can also fine tune and refine LLM outputs according to my needs, I am very well aware of context engineering and can deliver projects without being a vibe coder. I have understanding of optimization algorithms, CNNs, Generative Deep learning architectures like GANs, diffusion based models for media generation.

2. What are the key architectural components to create a chatbot based on LLM? Please explain the approach on a high-level

   Solution:

   On a higher level, the LLM architecture can be explained as:

   User side interfacing:

   Whatever user inputs the text into the message area, it could be in a website, mobile app or even command line based. A user can select different modes based on extensions like research, web search, image creation etc.

   API Calling layer:

   It is the layer which calls and interacts with APIs gateways and backend, puts rate limiting to repeated queries, manages sessions based on IP address, windows and localstorage, forwarding requests to the RAG layers,LLM layer depending on the LLM chatbot.

   LLM Layer:

   This layer constructs the prompts using specific instructions specified by the user, it could be guardrails, temperature of the LLM responses, putting token

limits on chats, Langchain can be used on this layer for retrieving specific information or document information.

LLM Understanding layer:

This layer actually understands what the prompt is about, and decides what responses should be delivered to the user.

Knowledge layer:

This layer is also called RAG layer which stands for retrieval augmented layer (RAG) which is useful for document processing and retrieving necessary information, utilizes the useful information from the documents using python OCR libraries and converts the data into chunks and based on the cleaned and processed data it generated the vector embedding which is basically numerical values assigned to the tokens and distances are calculated for token prediction purposes, for storing the vector embeddings the vector databases are used.

Output layer:

The layer outputs the messages or answers to the user based on the prompt and vector embeddings.

3. Please explain vector databases. If you were to select a vector database for a hypothetical problem (you may define the problem) which one will you choose, and why?

I would still choose the PostgreSQL with pgvector extension for building a vector based database despite the popular alternatives like pinecone and vectorDB, because I'm more familiar with PostgreSQL, and it integrates well with the structured data as well